src/flightCompV2/sensorfusion.cpp

```cpp
/*
 * Program to combine all ARC-S Sensors into positional tracking/calculations.
 *
 * Name:      Thor Kopenkoskey
 * Created:   July 1, 2025
 * Edited:    Jan 27, 2025
 */

#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BusIO_Register.h>
#include <Adafruit_LIS3MDL.h>
#include <Arduino_LSM6DSOX.h>
#include <cmath>
#include <BMP388_DEV.h>
#include <sensorfusion.h>

  const double GRAVITY = 9.80665;
  craftState vehicle;
  Adafruit_LIS3MDL magnetometer;
  BMP388_DEV bmp388;

  float ax = 0, ay = 0, az = 0;
  float gx = 0, gy = 0, gz = 0;
  float t = 0, p = 0, a = 0;

void craftState::initalizeSensors()
{
  bmp388.begin();                                 // Default initialisation, place the BMP388 into SLEEP_MODE
  bmp388.setTimeStandby(TIME_STANDBY_1280MS);     // Set the standby time to 1.3 seconds
  bmp388.startNormalConversion();

  if (!IMU.begin())
  {
    Serial.println("IMU failed!");
  }
  else
  {
    Serial.println("IMU OK");
  }

  if (!magnetometer.begin_I2C(LIS3MDL_ADDR, &Wire))
  {
    Serial.println("Magnetometer failed!");
  }
```

```
  else
  {
    Serial.println("Mag OK");
  }
}

double getDeltaTime() {
    static uint32_t last = micros();
    uint32_t now = micros();

    uint32_t delta = now - last;
    if (delta == 0) return 0;

    last = now;
    return delta * 1e-6;
}


void craftState::readBarometerData()
{
  float t, p, a;
  if (!isfinite(vehicle.altitude_m) ||
      !isfinite(vehicle.pressure_Pa)) {
      Serial.print("0.00,0.00,0.00");
      return;
  }

  else if (bmp388.getMeasurements(t, p, a))
  {
    vehicle.temperature_C = t;
    vehicle.pressure_Pa   = p;
    vehicle.altitude_m    = a;
  }

}
void craftState::calibrateGyroBias()
{
    const int N = 100;
    Vec3 sum;

    for (int i = 0; i < N; i++)
    {
        float gx, gy, gz;
        if (IMU.gyroscopeAvailable())
        {
            IMU.readGyroscope(gx, gy, gz);
            sum.x += gx;
            sum.y += gy;
            sum.z += gz;
```

```
        }
        delay(10);
    }

    gyro_bias_dps.x = sum.x / N;
    gyro_bias_dps.y = sum.y / N;
    gyro_bias_dps.z = sum.z / N;

    Serial.println("Gyro bias calibrated:");
    Serial.print(gyro_bias_dps.x); Serial.print(", ");
    Serial.print(gyro_bias_dps.y); Serial.print(", ");
    Serial.println(gyro_bias_dps.z);
}
//GAM = Gyro, Accel, Magno
void craftState::readGAM()
{
  float ax, ay, az;
  float gx, gy, gz;

  if (IMU.accelerationAvailable())
  {
    IMU.readAcceleration(ax, ay, az);
  }
  if (IMU.gyroscopeAvailable())
  {
    IMU.readGyroscope(gx, gy, gz);
  }
  //WHERE YOU SWITCH OUT AXIS's BASED ON SENSOR POSITION, What is Vehicle's X compared to the senors X? Is Sensors X really Vehicles Y?
  //CONFORM TO VEHICLE.
  vehicle.accel_g.x = ax;
  vehicle.accel_g.y = ay;
  vehicle.accel_g.z = az;

  vehicle.gyro_dps.x = gx - gyro_bias_dps.x; //See bias function, remove gyro basis due too... whatever external consistent factors...
  vehicle.gyro_dps.y = gy - gyro_bias_dps.y;
  vehicle.gyro_dps.z = gz - gyro_bias_dps.z;

  sensors_event_t magEvent;
  magnetometer.getEvent(&magEvent);

  vehicle.mag_uT.x = magEvent.magnetic.x;
  vehicle.mag_uT.y = magEvent.magnetic.y;
  vehicle.mag_uT.z = magEvent.magnetic.z;
}

void craftState::duringFlightOrientationTracking(double dt)
{
```

```
142     // Integrate to get angles, LSOX Sensor gives rotation velocity
143     vehicle.gyroAngle_deg.x += vehicle.gyro_dps.x * dt;
144     vehicle.gyroAngle_deg.y += vehicle.gyro_dps.y * dt;
145     vehicle.gyroAngle_deg.z += vehicle.gyro_dps.z * dt;
146 }
147 
148 void craftState::duringFlightPositionalTracking(double dt)
149 {
150 
151   vehicle.velocity_mps.x += (vehicle.accel_g.x * GRAVITY * dt);
152   vehicle.velocity_mps.y += (vehicle.accel_g.y * GRAVITY * dt);
153   vehicle.velocity_mps.z += (vehicle.accel_g.z * GRAVITY * dt);
154 
155   vehicle.deadReckoningPosition_m.x += vehicle.velocity_mps.x * dt;
156   vehicle.deadReckoningPosition_m.y += vehicle.velocity_mps.y * dt;
157   vehicle.deadReckoningPosition_m.z += vehicle.velocity_mps.z * dt;
158 
159 
160 }
161 
162 void craftState::preFlightOrientationTracking(double dt)
163 {
164   const float alpha = 0.98;
165   //Corect pitch/roll with IMU while static
166   vehicle.gyroAngle_deg.x = alpha * vehicle.gyroAngle_deg.x + (1 - alpha) *
    (vehicle.pitch * 180.0 / PI);
167   vehicle.gyroAngle_deg.y = alpha * vehicle.gyroAngle_deg.y + (1 - alpha) *
    (vehicle.roll * 180.0 / PI);
168 
169   //Correct yaw with fusion Magnometer
170   vehicle.yaw = atan2(vehicle.mag_uT.y, vehicle.mag_uT.x) * (180.0 / PI); //
    degrees
171   if (vehicle.yaw < 0) vehicle.yaw += 360.0; // Normalize to 0-360
172 
173   vehicle.gyroAngle_deg.z = alpha * vehicle.gyroAngle_deg.z + (1 - alpha) *
    vehicle.yaw;;
174 }
175 
176 void craftState::preFlightPositionalTracking(double dt)
177 {
178   if (dt < 1e-6) return;
179 
180   vehicle.computeGravityWhileStationary(); //Gives Gravity IMU Offset
181 
182   vehicle.velocity_mps.x = (vehicle.accel_g.x * 9.81) - vehicle.gravity_mps2.x;
183   vehicle.velocity_mps.y = (vehicle.accel_g.y * 9.81) - vehicle.gravity_mps2.y;
184   vehicle.velocity_mps.z = (vehicle.accel_g.z * 9.81) - vehicle.gravity_mps2.z;
185 }
186 
187 void craftState::computeGravityWhileStationary()
```

```
{
    // Compute pitch and roll from accelerometer
    pitch = atan2(accel_g.x, sqrt(accel_g.y*accel_g.y + accel_g.z*accel_g.z));
    roll  = atan2(accel_g.y, sqrt(accel_g.x*accel_g.x + accel_g.z*accel_g.z));

     // Gravity vector in sensor frame (m/s²)
    gravity_mps2.x = 9.81 * sin(pitch);
     gravity_mps2.y = 9.81 * sin(roll);
     gravity_mps2.z = sqrt(9.81 * 9.81 - gravity_mps2.x*gravity_mps2.x - gravity_mps2.y * gravity_mps2.y);
}


```